Tätigkeitsbericht 2026

Eigenständige Entwicklungsprojekte in Elektronik, Automatisierung und Softwareentwicklung

rwtec.neocities.org · github.com/RwHexa · rwwer999@outlook.de · Berichtszeitraum Januar bis Oktober 2026 · Stand 22. Oktober 2026

Kurzprofil

Im Berichtszeitraum habe ich 32 eigene Entwicklungsprojekte begonnen, durchgeführt und dokumentiert. Die Bandbreite reicht von der Leistungselektronik – Schaltnetzteil, Linearnetzteil, NF-Verstärker, Motoransteuerung – über Steuerungs- und Regelungstechnik mit SPS und ESP32, Gebäudeautomation über MQTT und Industrie-Visualisierung bis zu Anwendungssoftware für Windows, Browser und Android. Jedes Projekt habe ich vollständig bearbeitet: Aufgabenstellung klären, Schaltung oder Programm entwerfen, aufbauen beziehungsweise umsetzen, in Betrieb nehmen, messen und prüfen, anschließend nachvollziehbar dokumentieren. Alle Projekte sind versionskontrolliert abgelegt, 25 davon öffentlich einsehbar.

32	12	25	72
PROJEKTE	TÄTIGKEITSFELDER	ÖFFENTLICH DOKUMENTIERT	SEITEN FACHBUCHREIHE

BESONDERES – EIGENE FACHBUCHREIHE

Netzteile verstehen

Drei Wege von 230 V Wechselspannung zur stabilen Gleichspannung · Sammelband aus drei Heften · 72 Seiten · RwTec 2026

Aus der Werkstattarbeit dieses Jahres ist eine eigene Lernreihe zur Leistungselektronik entstanden, die ich vollständig selbst verfasst, bebildert und gesetzt habe. Die drei Hefte stellen getaktete und lineare Netzteile nebeneinander und arbeiten heraus, warum derselbe Zweck auf völlig unterschiedlichen Wegen erreicht wird:

Heft 1	**Schaltnetzteil verstehen**	Sperrwandler, getaktet
Heft 2	**Trafo-Netzteil verstehen**	linear, MOSFET-Vorregler, plus/minus 15 V
Heft 3	**LNK304 verstehen**	Ein-Chip-Netzteil, Buck-Wandler, 12 V

Die Bauteile werden der Reihe nach erklärt – Sicherung, Transformator, Gleichrichter, Kondensator, Z-Diode, Transistor, MOSFET und Spannungsregler – jeweils mit den Formeln, die man dafür wirklich braucht, und mit Messwerten, die ich am eigenen Aufbau überprüft habe. Ein eigener Teil behandelt die zugehörige Leiterplatte als KiCad-Projekt. Die Reihe wendet sich an Einsteiger, die die Zusammenhänge suchen, ebenso wie an Erfahrene, die ihr Wissen ordnen möchten, und ist so geschrieben, dass sich jede Schaltung nachbauen und nachmessen lässt.

Tätigkeitsfelder

1 Leistungselektronik und Schaltungsentwicklung

Schwerpunkt des Jahres. Einen 12-V-Sperrwandler mit dem Regler-IC OB2263 habe ich vollständig durchgearbeitet: Funktionsweise analysieren, Dimensionierung nachrechnen, Schaltung simulieren, am Aufbau messen und die Ergebnisse als Werkstattbericht festhalten. Ein Linearnetzteil von 230 V Wechselspannung auf geregelte +12 V (7812) habe ich als komplettes Elektronik-CAD-Projekt bis zu den Fertigungsdaten entwickelt. Hinzu kamen ein NF-Verstärker mit Aufbau und Messung sowie die elektronische Ansteuerung von Drehstrommotoren, die ich über mehrere Monate fortgeführt habe.

Technik: *Sperrwandler OB2263, Linearregler 7812, Netz- und Kleinspannungstechnik, KiCad (Schaltplan, Layout, Fertigungsdaten), Schaltungssimulation, Messtechnik*

2 Steuerungs- und Regelungstechnik

SPS-Programmierung mit OpenPLC 4.2.9, darauf aufbauend eine Zeitschaltuhr mit Bedienung und Anzeige über ein Nextion-Bedienpanel. Als Regelungsaufgabe habe ich die Füllstandsregelung eines Tanks mit PID-Regler auf dem ESP32 umgesetzt und zur Parametrierung eine Online-Simulation ergänzt, mit der sich das Regelverhalten vor dem Eingriff in die Anlage beurteilen lässt.

Technik: *OpenPLC 4.2.9, SPS-Programmierung, PID-Regelung, Regelstreckensimulation, ESP32 unter Arduino, Nextion-HMI*

3 Mikrocontroller und eingebettete Systeme

Firmware in C und C++ für ESP32 und ESP32-S3. Dazu gehören ein Webserver, der das Gerät selbst als Access-Point betreibt und die Bedienoberfläche ausliefert, eine serielle Anzeige- und Bedieneinheit zwischen Controller und Display sowie ein Raddisplay mit 4,2 Zoll großem PLCD, das ich gezielt auf gute Ablesbarkeit bei Sonneneinstrahlung ausgelegt habe; den Layout-Entwurf habe ich vorab in Delphi erstellt.

Technik: *ESP32, ESP32-S3, C/C++, Arduino-Framework, serielle Schnittstellen, Display-Ansteuerung, PLCD, Nextion*

4 SmartHome und Gebäudeautomation

Über das Jahr hinweg ist in mehreren Ausbaustufen eine durchgängige SmartHome-Lösung entstanden. Kern ist eine selbst geschriebene Desktop-Anwendung in Lazarus / Free Pascal, die Geräte über MQTT zentral steuert und visualisiert; als Broker dient die HiveMQ Cloud, die Sensorik läuft auf dem ESP32. Angebunden sind Shelly-Geräte mit Leistungs-, Spannungs- und Strommessung in Echtzeit, eine Rollosteuerung mit Positionsanzeige sowie ein ESP32-Sensorknoten für Temperatur und Luftfeuchte mit schaltbarem Ausgang; ein Live-Log zeigt alle MQTT-Meldungen der Geräte mit. Parallel dazu habe ich eine Browser-Variante gebaut, die plattformübergreifend und ohne Installation läuft: 3D-Grundriss mit anklickbaren Räumen, Live-Verbrauchsdaten in Watt, Volt, Ampere und Kilowattstunden sowie zeitbasierte Automatisierung – bewusst ohne Cloud-Zwang und ohne Abonnement.

Technik: *MQTT, HiveMQ Cloud, Shelly 1PM Mini und Gen3, Lazarus / Free Pascal, ESP32-Sensorik (Temperatur, Luftfeuchte), Rollosteuerung, Energiemessung, Browser-Oberfläche*

5 Funktechnik und Fernübertragung

Eine bidirektionale LoRa-Funkstrecke auf 868 MHz zwischen zwei ESP32 über E90-DTU-Modems habe ich mit eigenem Protokoll, Quittungsbetrieb und Failsafe-Verhalten aufgebaut und dokumentiert. Als durchgängige Anwendung entstand daraus ein Fertigmelder für die Waschmaschine: Eine selbst entworfene Platine misst über einen Hall-Stromsensor mit galvanischer Trennung die Leistungsaufnahme, die ESP-Steuerung unterscheidet über eine Schwellwertlogik die Zustände Bereit, Läuft und Fertig und meldet das Waschende per LoRa an einen Empfänger, der die Nachricht weiterreicht.

Technik: *LoRa 868 MHz, E90-DTU-Modems, Übertragungsprotokoll mit Quittung und Failsafe, ACS712-Hall-Stromsensor, Optokopplertrennung, Schwellwertlogik, Push-Benachrichtigung*

6 Industrie-Visualisierung und Prozessanzeige

Für die Anzeige eines Silo-Füllstands habe ich ein eigenes PC-Programm geschrieben, das den Messwert in Echtzeit von einer externen Steuerung – etwa einem ESP32 – über die serielle USB-Schnittstelle übernimmt und doppelt darstellt: grafisch im Silobild und zusätzlich als Prozentwert. Die Verbindung wird über eine einfache COM-Port-Auswahl hergestellt. Neben dem Sensorbetrieb gibt es einen manuellen Testbetrieb über ein Potentiometer, mit dem sich Anzeige und Skalierung ohne angeschlossene Anlage prüfen lassen. Ausgeliefert wird das Programm als einzelne ausführbare Datei ohne Installation – ein Punkt, der an Anlagenrechnern mit eingeschränkten Rechten regelmäßig den Ausschlag gibt.

Technik: *Serielle Kommunikation über USB und COM-Port, Echtzeitvisualisierung, Delphi-/Lazarus-Anwendung, ESP32 als Messwertgeber, installationsfreie Auslieferung*

7 Anwendungsentwicklung mit Delphi und Lazarus

Delphi und Lazarus sind die Grundlage mehrerer Anwendungen dieses Jahres. Die beiden größten – die Turnierverwaltung als VCL-Anwendung und das Abrechnungsprogramm für die Eisdiele – beschreiben die beiden folgenden Felder. Daneben entstanden MQTT-Clients unter Delphi und Lazarus, die Bedienoberfläche der SmartHome-Steuerung, das Programm zur Silo-Visualisierung sowie Display-Layouts, die ich in Delphi entworfen und anschließend auf die Hardware übertragen habe.

Technik: *Delphi VCL, Lazarus / Free Pascal, Object Pascal, TMS Web Core, MQTT-Anbindung*

8 Web- und App-Entwicklung

Mehrere Anwendungen habe ich bewusst serverlos als reine Browseranwendung umgesetzt, damit sie ohne Installation und ohne Internetzugang am Einsatzort laufen; die beiden größten – die Turnierverwaltung und die Kasse für die Eisdiele – sind in den folgenden Feldern gesondert beschrieben. Hinzu kommen eine Shelly-Oberfläche für Android, eine mit MIT App Inventor erstellte Android-App mit Anbindung an den Controller und das Hosting eigener Seiten.

Technik: *HTML, CSS, JavaScript, TypeScript, TMS Web Core, GitHub Pages, MIT App Inventor, Webhosting*

9 Turnierverwaltung für den Vereinssport

Die Turnierverwaltung ist über das Jahr in drei Fassungen gewachsen. Den Anfang machte eine Delphi/VCL-Anwendung für Windows mit Gruppenphase, KO-Runde, Tabellen und Ausdrucken. Darauf folgten zwei Browserfassungen, die ohne Installation und ohne Server auskommen und deshalb auch vom USB-Stick, vom Tablet oder vom Handy laufen – wichtig, weil auf dem Sportplatz selten verlässliches Netz zur Verfügung steht; die Daten bleiben dabei auf dem Gerät. Der Turniermodus bildet den Vereinsalltag ab: zwei Gruppen mit je bis zu acht Mannschaften, in der Gruppe jeder gegen jeden, Erste und Zweite über Kreuz ins Halbfinale, dazu Spiel um Platz 3, Finale und Elfmeterschießen, wenn ein K.-o.-Spiel unentschieden endet. Die beiden Tabellen sortieren nach Punkten, Tordifferenz und Toren; Halbfinalpaarungen und Endstand rechnet das Programm nach jedem Ergebnis neu aus, sodass kein Zwischenstand veralten kann.

Für den Turniertag selbst habe ich die Bedienung auf Tempo ausgelegt. Offene Begegnungen stehen als Liste bereit und sind mit einem Klick gesetzt; die Eingabe erkennt selbst, ob daraus ein Gruppenspiel oder das passende K.-o.-Spiel wird, und sagt es an, bevor das erste Tor fällt. Für die Beamerprojektion gibt es eine Live-Anzeige mit laufendem und nächstem Spiel, die sich auf Vollbild schalten lässt, sodass die Zuschauer nur den Spielstand sehen; Tore, Rücknahmen, Spielwechsel und Vollbild laufen über Tastenkürzel, damit am Anzeigerechner niemand nach Schaltflächen sucht. Zum Abschluss druckt das Programm einen Aushang auf einer A4-Seite mit beiden Tabellen, der K.-o.-Runde und dem Endstand. Turniere lassen sich zwischen PC und Handy sichern und laden, und ein Turnier aus der älteren Webfassung kann übernommen werden. Die Fassung mit zwei Gruppen ist quelloffen unter MIT-Lizenz veröffentlicht und lässt sich unter rwhexa.github.io/FussballGruppen mit geladenen Demo-Mannschaften in zwei Klicks ausprobieren. Eine Variante für Handballturniere und eine digitale Turnier-Karte mit Gruppentabelle und automatischer Platzierung ergänzen die Familie.

Technik: *HTML, CSS, JavaScript, Delphi/VCL, Betrieb ohne Server und ohne Netz, lokale Datenhaltung mit Sichern und Laden, Beamer-Vollbildanzeige mit Tastatursteuerung, Druckausgabe als A4-Aushang, GitHub Pages, MIT-Lizenz*

10 Kassensystem für die Gastronomie

Aus der Abrechnung für eine Eisdiele ist über drei Ausbaustufen eine vollständige Touch-Kasse geworden: zuerst eine Delphi-Anwendung, dann die Umsetzung als reine Browseranwendung, zuletzt eine Fassung mit Zugriffsschutz und gesicherter Datenhaltung. Die Kasse „San Remo“ läuft ohne Installation im Browser auf Windows-PC und Android-Tablet und verwaltet bis zu acht Tische neben Theke und Mitnahme, jeden mit eigenem Warenkorb; Nachbestellungen addieren sich selbsttätig, eine farbige Übersicht zeigt alle offenen Tische. Die Mehrwertsteuer bildet die Kasse nach der tatsächlichen Regel: 7 Prozent bei Mitnahme, 19 Prozent vor Ort, pro Tisch umschaltbar, Getränke ausnahmslos mit 19 Prozent – genau die Stelle, an der im Tagesgeschäft die meisten Fehler entstehen. Kassiert wird bar oder mit Karte, die Tischsumme wird nach Netto, Steuer und Brutto aufgeschlüsselt.

Der zweite Schwerpunkt lag auf dem Betrieb unter realen Bedingungen. Nach jeder Änderung wird der Stand aller offenen Tische gespeichert, sodass ein Stromausfall oder ein Browser-Neustart keine Bestellung kostet. Auf dem Tablet läuft die Kasse im PIN-gesicherten Vollbild-Kiosk, startet beim Einschalten von selbst und hält den Bildschirm wach. Ausgeliefert wird über GitHub Pages, womit jedes Gerät ohne Zutun auf dem aktuellen Stand ist. Damit der Betreiber sein Sortiment selbst pflegen kann, habe ich einen Artikel-Editor ergänzt: Artikel lassen sich in einer Tabelle anlegen und ändern, nach Kategorie filtern und durchsuchen; der Editor lädt den Bestand aus dem Repository und schreibt ihn – mit Sicherung der Vorfassung – wieder zurück. Sortimentspflege also ohne eine Zeile Code. Ausprobieren lässt sich die Kasse ohne Anmeldung unter rwhexa.github.io/eisdielerw. Für den Echtbetrieb ist zusätzlich eine technische Sicherheitseinrichtung nach Kassensicherungsverordnung vorgeschrieben; die Anbindung einer Cloud-TSE ist ebenso vorgesehen wie Bondruck, Tagesabschluss mit CSV-Export für den Steuerberater und der Betrieb mehrerer Kassen.

Technik: *HTML, CSS, JavaScript, Delphi/VCL (Erstfassung), Mehrwertsteuerlogik, GitHub Pages, GitHub-Anbindung des Artikel-Editors, lokale Datenhaltung mit Wiederanlauf, Fully Kiosk Browser unter Android, Kassensicherungsverordnung (TSE)*

11 Konstruktion und Fertigung

Konstruktion eigener Bauteile mit CAD und Aufbereitung für den 3D- und Metalldruck, einschließlich der Frage, welche Geometrien sich im jeweiligen Verfahren sinnvoll fertigen lassen.

Technik: *CAD-Konstruktion, 3D-Druck, Metalldruck, KI-gestützte Konstruktion*

12 Arbeitsweise, Dokumentation und Werkzeuge

Alle Projekte sind durchgängig mit Git versioniert und auf GitHub abgelegt. Zu mehreren Themen habe ich über die reine Funktion hinaus Lehr- und Werkstattberichte geschrieben, die Herleitung, Messwerte und Prüfschritte offenlegen, sodass sie nachvollzogen und wiederholt werden können. Den Einsatz von KI als Entwicklungswerkzeug habe ich dabei systematisch erprobt und die daraus entstandene Werkzeugkette für die Delphi-Entwicklung schriftlich festgehalten; ein Netzteilprojekt ist vollständig auf diesem Weg entstanden. Den Abschluss dieser Arbeit bildet die eingangs genannte Fachbuchreihe, in der ich die Ergebnisse aus mehreren Netzteilprojekten zu einem geschlossenen Sammelband zusammengeführt habe.

Technik: *Git, GitHub, GitHub Pages, Markdown-Dokumentation, Satz und Layout der Buchreihe, KI-gestützte Entwicklung*

Eingesetzte Technologien und Werkzeuge

Programmiersprachen	Object Pascal (Delphi, Lazarus), C, C++, JavaScript, TypeScript, Python, HTML, CSS
Mikrocontroller und Hardware	ESP32, ESP32-S3, Arduino-Framework, Nextion-Display, PLCD 4,2 Zoll, Shelly 1PM Gen3, E90-DTU-LoRa-Modem
Automatisierung und Regelung	OpenPLC 4.2.9, SPS-Programmierung nach IEC 61131-3, PID-Regelung, Regelstreckensimulation
Vernetzung und Visualisierung	MQTT mit HiveMQ Cloud, serielle Kommunikation über USB und COM-Port, Echtzeitvisualisierung, Energiemessung, Browser-Oberflächen
Elektronik-CAD und Messtechnik	KiCad (Schaltplan, Leiterplattenlayout, Fertigungsdaten), Schaltungssimulation, Messungen am Aufbau
Konstruktion	CAD, 3D-Druck, Metalldruck
Frameworks und Werkzeuge	TMS Web Core, MIT App Inventor, Git, GitHub, GitHub Pages

Projektübersicht 2026 im zeitlichen Verlauf

Die folgende Aufstellung nennt alle 32 Projekte in der Reihenfolge ihrer Entstehung. Projekte ohne Vermerk sind öffentlich unter github.com/RwHexa einsehbar.

Jan 2026	**soccer**	Einstiegsprojekt React Native (Expo) – nicht weitergeführt (privat)
Mär 2026	**ESP32Webserver**	Webserver auf dem ESP32 im Access-Point-Betrieb (privat)
Mär 2026	**MqttLazarus**	MQTT-Anbindung an den Broker mit Lazarus-Oberfläche
Mär 2026	**MqttDelphi**	MQTT-Client unter Delphi und Lazarus (privat)
Mär 2026	**SmartHome-Extern**	Betrieb der SmartHome-Umgebung unter Windows (privat)
Apr 2026	**gitrepo**	Einarbeitung in die Versionskontrolle (privat)
Apr 2026	**shelly-controller**	Shelly-Steuerung über MQTT, Oberfläche mit TMS Web Core
Apr 2026	**shellyweb**	Shelly-Bedienoberfläche für Android
Mai 2026	**shwindows**	SmartHome-Bedienung unter Windows
Mai 2026	**eisdiele**	Abrechnung als Delphi-Anwendung – erste Fassung
Mai 2026	**eisdielerw**	Touch-Kasse „San Remo“ als reine Browseranwendung
Mai 2026	**eisdielezw**	Ausbaustufe: Zugriffsschutz, Datensicherung, Artikel-Editor
Mai 2026	**motordrehzahl**	Elektronische Ansteuerung von Drehstrommotoren (bis Oktober)
Jun 2026	**konstruktionrw**	CAD-Konstruktion für 3D- und Metalldruck
Jun 2026	**nf_verstaerker**	NF-Verstärker: Schaltung, Aufbau und Messung
Jul 2026	**DelphiSkills**	Werkzeugkette für KI-gestützte Delphi-Entwicklung (privat)
Jul 2026	**FussballTurnier**	Turnierverwaltung als Delphi-VCL-Anwendung mit Beamer-Turnierbaum
Jul 2026	**NetzgeraetRw**	12-V-Sperrwandler OB2263 – Analyse, Prüfung, Simulation, Bericht

Jul 2026	**GeminiNetz**	Entwurf zum Schaltnetzteil (privat)
Jul 2026	**shelly-waschmaschine-lora-ntfy**	Fertigmelder: Strommessung, LoRa-Übertragung, Benachrichtigung
Jul 2026	**projektuebersicht**	Gesamtübersicht aller Repositories 2025/2026
Jul 2026	**SpsSteuerung**	SPS-Steuerung mit OpenPLC 4.2.9
Aug 2026	**zeitschaltuhr**	Zeitschaltuhr in der SPS mit Nextion-Bedienpanel
Aug 2026	**DisplayAnzBedien**	Anzeigen und Bedienen mit dem ESP32 über serielle Verbindung
Aug 2026	**RadDisplay**	Raddisplay PLCD 4,2 Zoll mit ESP32-S3, sonnenlichttauglich
Sep 2026	**FtmanagerCard**	Webhosting von Ftmanager und Card
Sep 2026	**KiCadPlatine**	Netzteil 230 V auf +12 V (7812): Schaltplan, Layout, Fertigungsdaten
Sep 2026	**AppInventorToController**	Android-App mit MIT App Inventor, Anbindung an den Controller
Okt 2026	**FussballTurnierWeb**	Turnierverwaltung im Browser, Live-Anzeige und Druckansicht
Okt 2026	**LoraRw**	Bidirektionale LoRa-Funkstrecke 868 MHz mit Protokoll und Failsafe
Okt 2026	**FussballGruppen**	Zwei Gruppen, K.-o.-Runde, Beamer-Anzeige, Aushang – Open Source
Okt 2026	**TankPID**	PID-Füllstandsregelung eines Tanks mit ESP32 und Simulation

Alle genannten Projekte sind unter **github.com/RwHexa** abgelegt; die öffentlichen Repositories enthalten Quelltext, Schaltpläne beziehungsweise Dokumentation und können auf Wunsch im Gespräch gezeigt werden. Die Fachbuchreihe **Netzteile verstehen** stützt sich auf die Netzteilprojekte des Jahres (NetzgeraetRw, KiCadPlatine, GeminiNetz) und liegt als Sammelband mit 72 Seiten vor; sie wird auf Wunsch gern vorgelegt. Einzelne Arbeiten führe ich nicht als eigenes Repository, sondern auf der unten genannten Portfolioseite – darunter die Silo-Füllstandsvisualisierung und die Handball-Variante der Turnierverwaltung.

AUSGEWÄHLTE PROJEKTE ONLINE

rwtec.neocities.org

Die Projekte, die sich zeigen lassen, habe ich auf einer eigenen Portfolioseite aufbereitet – jeweils mit Kurzbeschreibung, Bildern, Dokumentation als PDF und, wo es die Technik zulässt, einer lauffähigen Demo zum Ausprobieren im Browser. Vertreten sind dort die Turnierverwaltung für Fußball und Handball, das ESP32-Webserver-Tableau, das Nextion-Display, das E-Bike-Raddisplay, die SmartHome-Steuerung über MQTT, die Silo-Füllstandsvisualisierung, die SPS-Steuerung mit OpenPLC, die Platine „Waschmaschine fertig“, die Lernreihe zum Schaltnetzteil, die Browser-Kasse „San Remo“ und eine KI-generierte Android-App.



Startseite der Portfolioseite rwtec.neocities.org